BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)

COLLEGE OF ENGINEERING, PUNE – 411043

A PROJECT BASED LEARNING

REPORT ON

EMAIL SPAM DETECTOR
(EMAIL SECURITY ANALYZER)

Submitted by

Name	PRN	Roll No.
Priyadarshi Dey	2414110202	18

For the course: Data Warehousing and Mining

Submitted To Course Coordinator

Dr. Rohini Khalkar

B. Tech Computer Engineering (Sem V)

2026–27



DEPARTMENT OF COMPUTER ENGINEERING

Bharati Vidyapeeth (Deemed to be University)

College of Engineering, Pune – 411043



CERTIFICATE

This is to certify that the project report entitled “Email Spam Detector (Email Security Analyzer)” is a bonafide work carried out by Priyadarshi Dey (18) in partial fulfilment of the requirements for the award of the degree of B. Tech in Computer Engineering during the academic year 2026–27. The work has been completed under my guidance and supervision. To the best of my knowledge, the work presented in this report has not been submitted, either in whole or in part, to any other university or institution for the award of any degree or diploma.

Date:

Dr. Rohini Khalkar
Internal Guide
Dept. of Computer Engineering

Dr. Sandeep Vanjale
Head of Department
Dept. of Computer Engineering

ACKNOWLEDGEMENT

I express my sincere gratitude to Dr. Rajesh Prasad, Principal of Bharati Vidyapeeth (Deemed to be University), College of Engineering, Pune, for providing the facilities and academic environment required for completing this project.

I am deeply thankful to Dr. Sandeep Vanjale, Head of the Department of Computer Engineering, for valuable guidance and encouragement throughout the project work. I extend my heartfelt gratitude to my project guide, Dr. Rohini Khalkar, for continuous guidance, constructive suggestions, technical support, and motivation during every stage of the project.

I also thank all faculty members, friends, and family members who supported me during the development and documentation of the Email Spam Detector. Finally, I thank my institution for providing an opportunity to undertake this project and apply concepts of data mining, text classification, and web development to a practical email-security application.

Priyadarshi Dey

TABLE OF CONTENTS

CERTIFICATE 2

ACKNOWLEDGEMENT 3

TABLE OF CONTENTS 4

ABSTRACT 6

1. INTRODUCTION 7

1.1 Background 7

1.2 Problem Statement 7

1.3 Objectives 7

1.4 Scope 8

1.5 Need for the System 8

1.6 Organization of the Report 8

2. LITERATURE SURVEY 9

2.1 Introduction 9

2.2 Spam Filtering as Text Classification 9

2.3 TF-IDF Representation 9

2.4 Naive Bayes Classification 9

2.5 Evaluation of Classifiers 9

2.6 Research Gap 10

3. REQUIREMENT ANALYSIS AND FEASIBILITY STUDY 11

3.1 Existing System 11

3.2 Proposed System 11

3.3 Functional Requirements 11

3.4 Non-Functional Requirements 12

3.5 Feasibility Study 12

3.6 Use-Case Description 12

4. SYSTEM DESIGN 13

4.1 System Architecture 13

4.2 Data-Flow Description 13
4.3 Data and Model Design 14
4.4 User-Interface Design 14
4.5 Scoring and Analytics Design 15
5. METHODOLOGY AND IMPLEMENTATION 16
5.1 Development Methodology 16
5.2 Data Collection 16
5.3 Data Preprocessing 16
5.4 Model Training and Evaluation Workflow 17
5.5 Algorithm Description 17
5.6 Modules of the System 18
5.7 URL Analysis 18
5.8 Technology Stack 18
5.9 Feature List 19
5.10 Implementation Details 19
6. TESTING AND RESULTS 24
6.1 Testing Strategy 24
6.2 Test Cases 24
6.3 Evaluation Metrics 24
6.4 Results and Discussion 25
7. CONCLUSION AND FUTURE SCOPE 27
7.1 Conclusion 27
7.2 Limitations 27
7.3 Future Scope 27
REFERENCES 29

ABSTRACT

Unsolicited bulk email (spam) and phishing messages waste users' time and trick them into revealing passwords, money, or personal details. A bare "spam / not spam" label is hard to trust, because the user cannot see why a message was flagged. The Email Spam Detector, extended into an Email Security Analyzer, is a Flask-based web application that classifies an email as spam or not spam using a machine-learning model and explains the result in a form a non-expert can follow.

The classifier is a pipeline of TF-IDF vectorization (unigrams and bigrams, English stop-word removal, up to 5,000 features) followed by a Multinomial Naive Bayes model from scikit-learn. For every email the system reports the spam probability, a confidence level, and a 0–100 risk score divided into Safe, Suspicious, and High-risk bands. A separate, clearly labelled rule-based layer highlights suspicious keywords (urgency, prize/money, account/credential, and promotional language), analyses links found in the text, computes text statistics, and produces a recommendation. Users can paste text or upload .txt / .eml files, review a dashboard and history of past analyses, and download a PDF security report.

The model was trained on a 30-message demonstration dataset (13 spam, 17 not spam). On a 20% hold-out set (6 messages) it achieved 83.33% accuracy; under 5-fold stratified cross-validation over all 30 messages it achieved 86.67% accuracy, 90.91% precision, 76.92% recall, and an F1-score of 83.33%. Because the dataset is very small, these figures are indicative only, and the report discusses this limitation openly.

Keywords: spam detection, text classification, TF-IDF, Naive Bayes, phishing, data mining, Flask, scikit-learn, email security.

1. INTRODUCTION

1.1 Background

Email is a primary channel for personal and business communication, which also makes it a primary target for spam, scams, and phishing. Spam messages typically advertise products, announce fake prizes, or imitate banks and online services to harvest credentials. Rule-based filters alone are easy to evade, whereas data-mining techniques can learn which words and phrases distinguish spam from legitimate mail from labelled examples.

Text classification is a standard data-mining task: a collection of labelled documents is converted into numeric features, a model is trained on those features, and the model then assigns a class to unseen documents. Spam detection is one of the best-known applications of this task.

1.2 Problem Statement

Users receive large amounts of email and often cannot judge quickly whether a message is legitimate. Simple detectors return only a label, which gives no evidence, no measure of certainty, and no guidance on what to do next.

The central problem addressed by this project is: How can a web application classify an email as spam or not spam with a trained model, and present the result together with a probability, a risk score, supporting evidence, link analysis, and a downloadable report so that users can understand and trust the outcome?

1.3 Objectives

- To build a machine-learning model that classifies messages as spam or not spam using TF-IDF and Multinomial Naive Bayes.
- To evaluate the model with accuracy, precision, recall, F1-score, a confusion matrix, and cross-validation, and display the results in the application.
- To present spam probability, confidence, and a 0–100 risk score for every analysed email.
- To explain results through influential terms, keyword highlighting, URL checks, and plain-language recommendations.
- To support pasted text and .txt / .eml uploads, a dashboard, a searchable history, and PDF report export.
- To provide a responsive web interface with a sidebar/navbar layout and dark and light themes.

- To demonstrate data-mining concepts (preprocessing, feature extraction, classification, evaluation) in a practical application.

1.4 Scope

The system is an educational prototype for email-content analysis. It analyses the text of a message that the user supplies. It does not connect to a mailbox, inspect email headers for authentication (SPF, DKIM, DMARC), open attachments, visit links, or consult blocklists. Analysis history is stored in the user's own browser, not in a database.

1.5 Need for the System

Phishing and spam continue to cause financial loss and security incidents. A tool that explains why an email looks risky helps users build good habits, such as not clicking unknown links and verifying the sender, and also demonstrates how data-mining techniques are applied to a real problem.

1.6 Organization of the Report

Chapter 1 introduces the project. Chapter 2 presents the literature survey. Chapter 3 discusses requirements and feasibility. Chapter 4 explains system design. Chapter 5 describes methodology and implementation. Chapter 6 presents testing and results. Chapter 7 concludes the project and outlines limitations and future enhancements.

2. LITERATURE SURVEY

2.1 Introduction

This chapter reviews the concepts behind the project: text representation with TF-IDF, probabilistic classification with Naive Bayes, and evaluation of classifiers.

2.2 Spam Filtering as Text Classification

Spam filtering is treated as binary classification over text. Each message is labelled spam or ham (legitimate), converted into a feature vector, and passed to a learning algorithm. Public collections such as the SMS Spam Collection (Almeida et al.) are widely used to benchmark such filters [6].

2.3 TF-IDF Representation

Term Frequency–Inverse Document Frequency weights a term by how often it occurs in a document and how rare it is across the collection, so words that are frequent in one message but uncommon overall receive a high weight [4]. Using bigrams in addition to single words captures short phrases such as “click here” or “free prize”.

2.4 Naive Bayes Classification

Naive Bayes applies Bayes’ theorem with the assumption that features are conditionally independent given the class. The multinomial variant models word counts or weights and is fast, needs little data, and is a long-standing baseline for text classification [2][5]. Its per-term log-probabilities also make it easy to explain which words pushed a message towards spam.

2.5 Evaluation of Classifiers

For spam detection the spam class is treated as positive. Precision measures how many messages flagged as spam really are spam; recall measures how many spam messages are caught; F1 is their harmonic mean. When data are scarce, k-fold cross-validation gives a more stable estimate than a single small hold-out split [1].

Table 1: Comparison of Common Spam-Detection Approaches

Approach	Basis	Strength	Limitation
Rule / keyword filters	Hand-written patterns	Transparent, no training needed	Easy to evade; needs manual upkeep
Naive Bayes + TF-IDF	Word statistics from labelled data	Fast, simple, explainable	Independence assumption; ignores word order
SVM / logistic regression	Learned linear boundaries	Often more accurate	Less directly interpretable
Deep learning (transformers)	Contextual embeddings	State-of-the-art accuracy	Needs large data and compute

2.6 Research Gap

Many student spam detectors output only a label. This project combines a trained classifier with an explanation layer (influential terms, highlighted keywords, URL checks, recommendation) and keeps the two clearly separated, so that rule-based findings are never mistaken for model output.

3. REQUIREMENT ANALYSIS AND FEASIBILITY STUDY

3.1 Existing System

Mail providers include their own filters, but a user who pastes a suspicious message into a typical simple detector receives only “spam” or “not spam”. There is no probability, no evidence, no link analysis, and no record of previous checks.

3.2 Proposed System

The proposed Email Security Analyzer accepts an email, runs the trained model, runs transparent rule-based checks, and presents the combined findings through a web interface with a dashboard, history, and PDF export.

3.3 Functional Requirements

Table 2: Functional Requirements

ID	Functional requirement
FR-01	The system shall accept email text pasted by the user.
FR-02	The system shall accept .txt and .eml uploads up to 1 MB.
FR-03	The system shall classify the email as spam or not spam using the trained model.
FR-04	The system shall display spam probability, confidence, and a 0–100 risk score with a risk band.
FR-05	The system shall list the terms that most influenced the prediction.
FR-06	The system shall highlight suspicious keywords and analyse links found in the text.
FR-07	The system shall show real-time statistics (characters, words, links, keywords and so on) while the user types.
FR-08	The system shall generate a downloadable PDF report.
FR-09	The system shall keep a searchable history and show a dashboard of past results.
FR-10	The system shall reject empty, oversized, or invalid input with a clear message.

3.4 Non-Functional Requirements

Table 3: Non-Functional Requirements

ID	Non-functional requirement
NFR-01	The interface should be simple and understandable for non-experts.
NFR-02	A result should be returned quickly for typical emails.
NFR-03	The model prediction and rule-based analysis must be visibly separated.
NFR-04	The code should be modular (routes, model service, heuristics, parser, report).
NFR-05	The application should work offline (charts library bundled locally).
NFR-06	The interface should be responsive and support dark and light themes.
NFR-07	Pasted and uploaded email content should not be stored on the server.

3.5 Feasibility Study

Technical feasibility: Python, Flask, scikit-learn, and pandas provide everything needed for training, prediction, and a web interface.

Economic feasibility: all tools used are open source and free, so the prototype can be built and run at no cost.

Operational feasibility: the interface needs no training; the user pastes an email and reads the result.

Schedule feasibility: the project was built incrementally in phases: model, web interface, explanation layer, reporting, testing, and documentation.

3.6 Use-Case Description

The primary actor is the user. The user opens the Analyzer page, pastes or uploads an email, and presses Analyze. The system returns the verdict, probabilities, evidence, and recommendation; the user may download the PDF report and later view the Dashboard and History pages. A developer or administrator may retrain the model by replacing spam.csv and running train_model.py.

4. SYSTEM DESIGN

4.1 System Architecture

The application follows a layered design with a clear separation between the machine-learning model and the rule-based explanation layer. The presentation layer contains pages, live statistics, charts, and history. The application layer validates input and combines results. The model service wraps the trained classifier, and the heuristics module performs transparent rule-based checks.

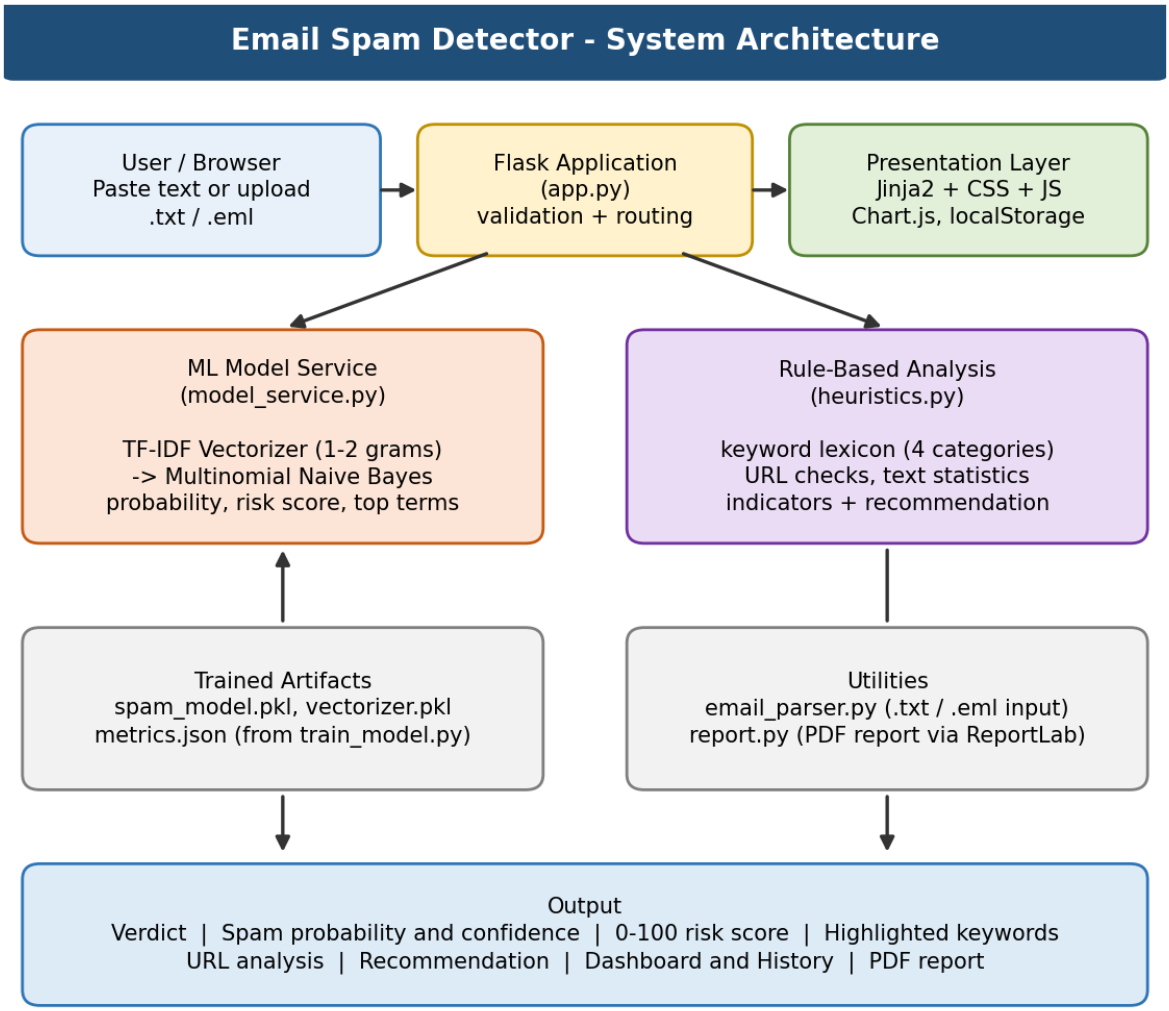


Figure 4.1: System Architecture

4.2 Data-Flow Description

1 The user pastes text or uploads a .txt / .eml file on the Analyzer page.

2 For uploads, the parser validates the extension, size, and content, and extracts the subject and body (HTML bodies are converted to text, keeping link targets).

3 The browser sends the text to the /api/analyze endpoint, which checks that it is not empty and is within 50,000 characters.

4 The model service vectorizes the text with the saved TF-IDF vectorizer and obtains class probabilities from Multinomial Naive Bayes.

5 The heuristics module independently scans for keywords, links, capitalization, and punctuation patterns.

6 The server returns one JSON response with a model section and a rules section, plus a verdict and recommendation.

7 The interface displays the result and saves it to the browser's local history; a PDF can be requested from /api/report, which re-analyses the text on the server.

4.3 Data and Model Design

The project uses a single labelled CSV file, spam.csv, with two columns: label (spam or ham) and message. During training, ham is mapped to 0 and spam to 1. There is no database; the trained model is stored as two pickle files, and evaluation results are stored in model/metrics.json.

Table 4: Dataset Summary (from model/metrics.json)

Item	**Value**
Total messages	30
Spam / not spam	13 / 17
Training / test split	24 / 6 (80% / 20%, random_state = 42)
Vocabulary size	167 terms
Features	TF-IDF, unigrams + bigrams, English stop words, max 5,000 features
Classifier	Multinomial Naive Bayes

4.4 User-Interface Design

The interface has a collapsible sidebar and a top navigation bar, with a dark and light theme. Each page has a defined purpose.

Table 5: Application Pages

Page	Route	Purpose
Home	/	Introduction and entry point
Analyzer	/analyzer	Paste or upload an email, live statistics, results, PDF download
Dashboard	/dashboard	Charts of spam vs safe, detections over time, risk distribution, detection categories, recent results, model metrics
History	/history	Searchable list of previous analyses (stored in the browser)
How It Works	/how-it-works	Pipeline explanation and evaluation results
About	/about	Project objective, features, technology, team information

4.5 Scoring and Analytics Design

Risk Score = round(P(spam) × 100)

Risk bands: 0–30 Safe, 31–60 Suspicious, 61–100 High risk. Confidence is the probability of the predicted class and is labelled High (80% or more), Medium (60–79%), or Low (below 60%).

The rule-based layer produces four breakdown factors from 0 to 100: keyword risk (18 points per high-severity and 8 per medium-severity keyword, capped at 100), URL risk (highest score among the links), urgency, and formatting. The overall rule-based level is High if any factor is 70 or more, Medium if 35 or more, otherwise Low. These rules never change the model's prediction.

5. METHODOLOGY AND IMPLEMENTATION

5.1 Development Methodology

The project followed an incremental development methodology. Each feature was developed and verified as a separate module before integration.

1 Build and evaluate the classifier (training script and metrics).

2 Wrap the model in a Flask backend with a JSON API.

3 Add the rule-based explanation layer (keywords, URLs, statistics).

4 Build the interface, sidebar/navbar, and dashboard.

5 Add file upload, history, and PDF report export.

6 Test, document, and prepare the report.

5.2 Data Collection

The dataset spam.csv contains 30 short labelled messages, such as lottery and prize announcements (spam) and everyday conversation (ham). It is a demonstration dataset, and its small size is a stated limitation. For stronger results it should be replaced by a larger public collection such as the SMS Spam Collection and the model retrained with train_model.py.

5.3 Data Preprocessing

- Labels are mapped to numbers (ham = 0, spam = 1) and rows with missing values are dropped.
- Text is converted to lower case and tokenized.
- English stop words are removed.
- Unigrams and bigrams are extracted, limited to the 5,000 most frequent features.
- TF-IDF weights are computed by the vectorizer, which is fitted on the training split only.

5.4 Model Training and Evaluation Workflow

The data are split 80/20 with a fixed random seed. The vectorizer is fitted on the training messages and a Multinomial Naive Bayes model is trained on the resulting matrix. The model and vectorizer are saved as pickle files. Because a six-message test set is very small, train_model.py also runs 5-fold stratified cross-validation using a pipeline (vectorizer + classifier), so that every message is used for testing once. All metrics, including confusion matrices, are computed by the script and written to model/metrics.json; the web pages read this file and no figure is typed in by hand.

5.5 Algorithm Description

1 Input: email text T.

2 Validate that T is non-empty and at most 50,000 characters.

3 Transform T into a TF-IDF vector using the saved vectorizer.

4 For each class, Multinomial Naive Bayes adds the log prior to the weighted log-likelihood of each term and converts the two scores to probabilities that sum to 100%.

5 Predict the class with the higher probability; compute risk score = round(P(spam) × 100) and the risk band.

6 Compute each term's contribution as its TF-IDF value × (log P(term | spam) − log P(term | ham)); report the top terms pushing towards spam and towards not spam.

7 Report vocabulary coverage (how many of the email's words are known to the model).

8 Run the rule-based checks on T and return both results together.

5.6 Modules of the System

Table 6: Modules

Module	File	Description
Web application	app.py	Page routes, JSON API (/api/analyze, /api/upload, /api/report, /api/model-info), /health, and error handling
Model service	services/model_service.py	Prediction, probabilities, risk level, confidence label, influential terms, vocabulary coverage, metrics access
Heuristics	services/heuristics.py	Keyword lexicon in four categories, URL analysis, statistics, indicators, breakdown, recommendation
Email parser	services/email_parser.py	Extraction from .txt and .eml files (1 MB limit, rejects binary content)
Report generator	services/report.py	Builds the PDF security report with ReportLab
Training script	train_model.py	Trains the model and writes model/metrics.json
Front end	templates/, static/	Interface, live statistics, dashboard charts, history

5.7 URL Analysis

Links found in the text are scored by simple rules: link shorteners (+40), raw IP address (+50), an "@" in the address (+40), no HTTPS (+15), uncommon top-level domains such as .xyz or .top (+25), punycode (+35), many sub-domains (+15), many hyphens (+15), a well-known brand name on a domain that is not the official one (+45), login-style words (+15), and very long addresses (+10). A score of 25 or more is labelled Suspicious; well-known domains with a low score are Safe-looking; others are Unknown. Links are never visited or checked against a blocklist, so these labels are not confirmed verdicts.

5.8 Technology Stack

Table 7: Technology Stack

Layer	Technology / implementation	Purpose
Language	Python 3	Backend and training
Web framework	Flask	Routes and JSON API
Machine learning	scikit-learn (TfidfVectorizer, MultinomialNB), pandas	Feature extraction, training, evaluation
Report	ReportLab	PDF security report
Front end	HTML5, CSS3, vanilla JavaScript, Jinja2	Interface and interaction
Charts	Chart.js (bundled locally)	Dashboard visualizations
Storage	Pickle files, JSON, browser localStorage	Model, metrics, analysis history

5.9 Feature List

Table 8: Feature List

Feature	Description
Email analysis	Classifies pasted text or uploaded .txt / .eml files
Live statistics	Characters, words, sentences, links, suspicious keywords, capitals, special characters, exclamation marks
Sample emails	Ready-made examples for quick demonstration
Risk meter	Spam probability, confidence label, and 0–100 risk score
Explanation	Influential terms and highlighted keywords (“Why this result?”)
URL analysis	Rule-based assessment of each link found
PDF report	Downloadable security report for each analysis
Dashboard and history	Charts of past results and a searchable history
Interface	Dark / light mode, collapsible sidebar, responsive layout

5.10 Implementation Details

The backend keeps the model and rule-based results in separate sections of one JSON response so that the interface can label them differently. The keyword lexicon and URL pattern are exported to the browser so that live statistics while typing match what the server later reports. History is stored in localStorage, so no email text is kept on the server. The PDF endpoint re-analyses the posted text on the server rather than trusting client-supplied results.

The following screenshots were captured from the running application using sample emails.

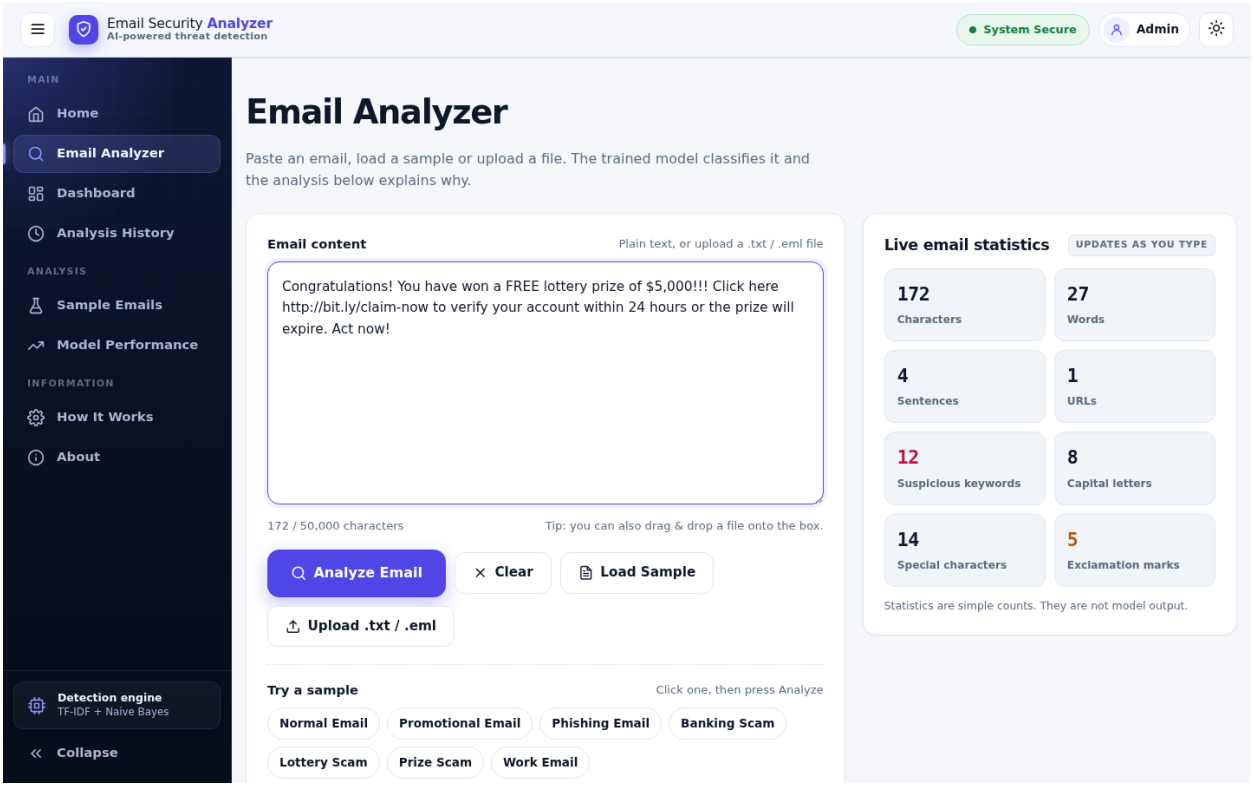


Figure 5.1: Analyzer page with sample email and live statistics

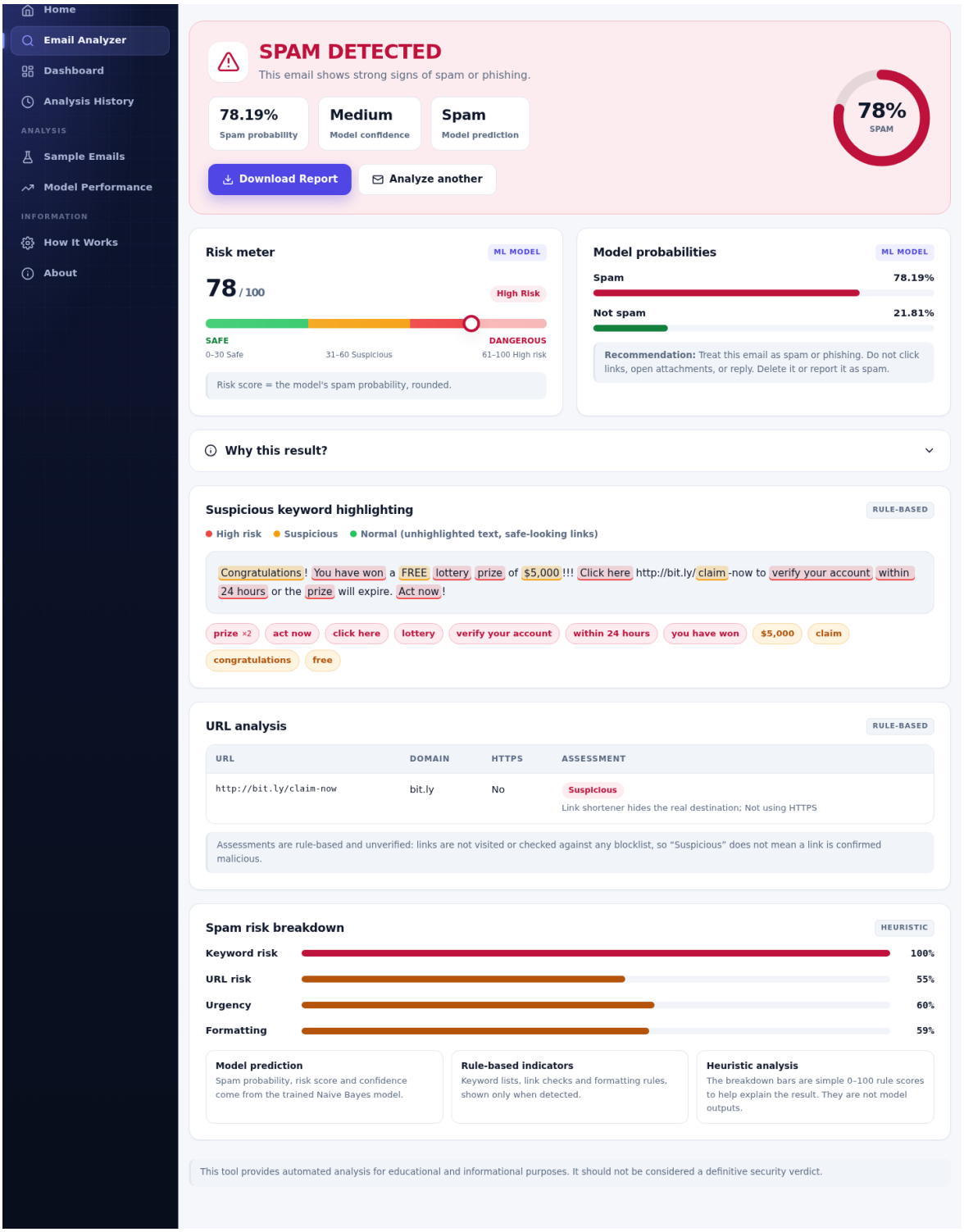


Figure 5.2: Analysis result showing verdict, risk meter, keyword highlighting, and URL analysis

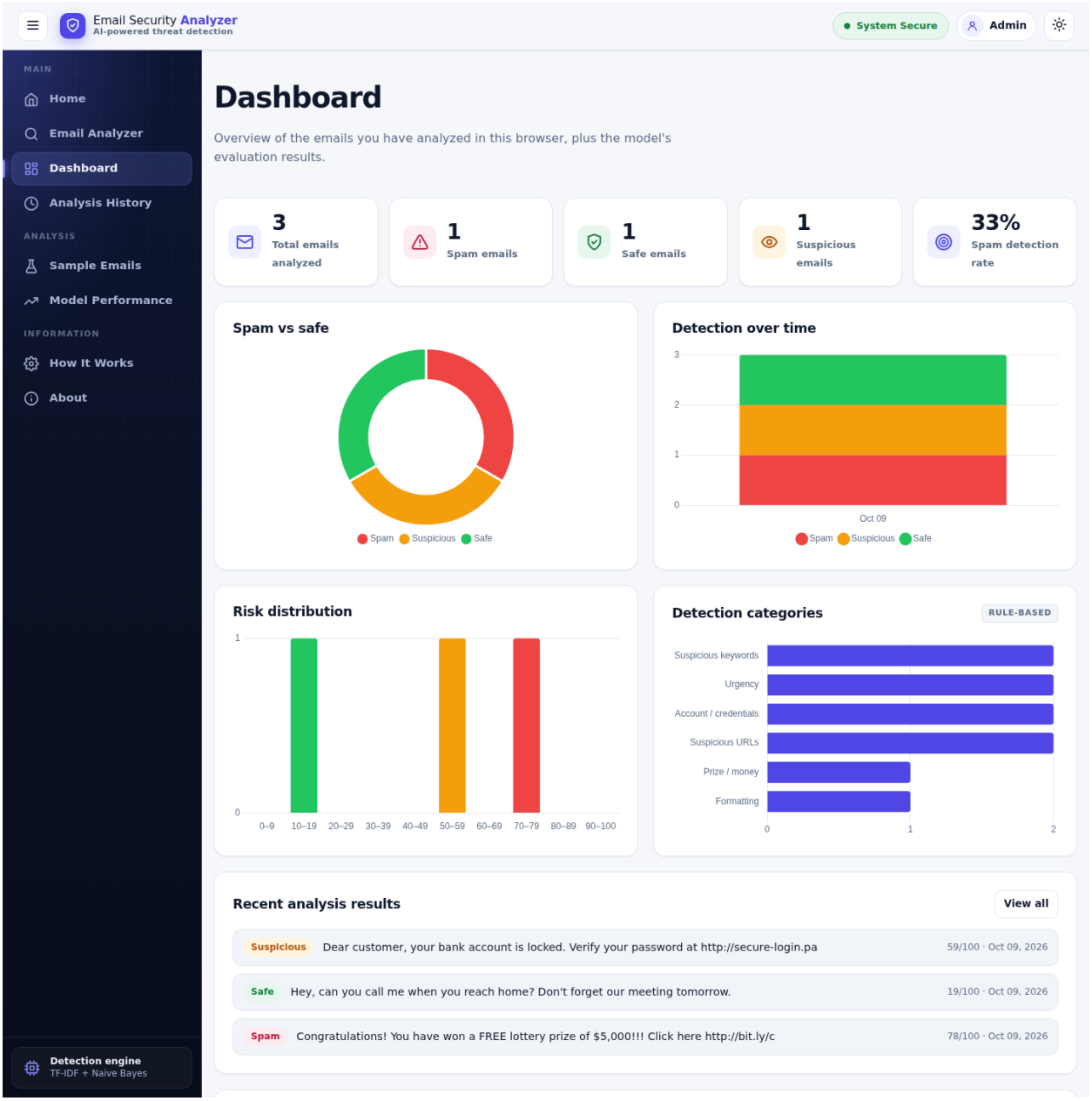


Figure 5.3: Dashboard with charts and recent analysis results

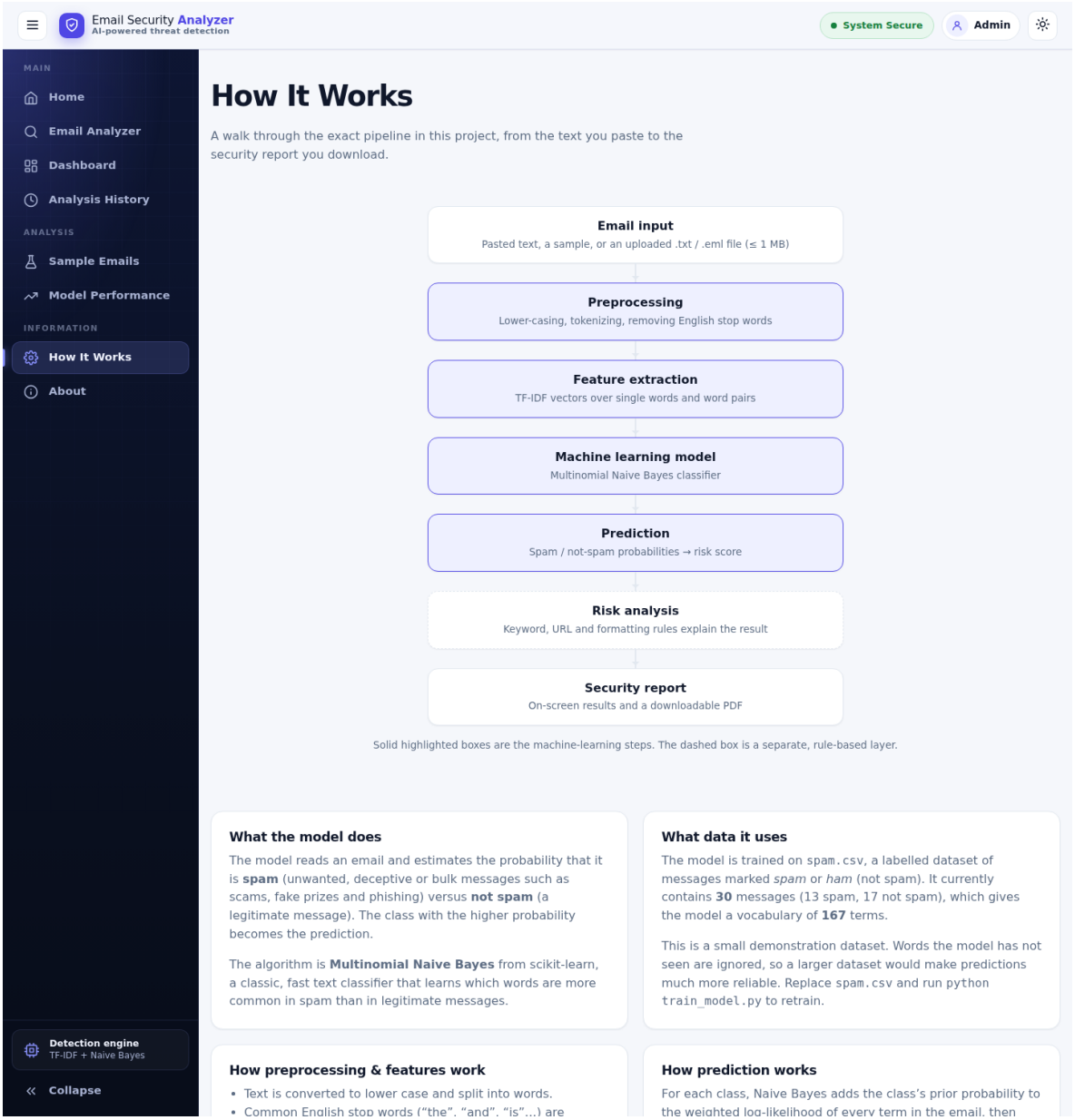


Figure 5.4: How It Works page

6. TESTING AND RESULTS

6.1 Testing Strategy

The application was tested through functional testing of the API endpoints, input-validation testing, page-availability checks, and evaluation of the classifier on held-out data. The endpoint tests below were executed against the project code using Flask's test client.

6.2 Test Cases

Table 9: Test Cases and Observed Results

ID	Scenario	Input / action	Expected result	Observed	Status
TC-01	Spam email	Lottery/prize text with a bit.ly link and urgent wording	Spam, high risk	Spam, 77.8% spam probability, High risk; link flagged Suspicious	Pass
TC-02	Legitimate email	"Can you call me when you reach home? Don't forget our meeting tomorrow."	Not spam, safe	Not spam, 19.25% spam probability, Safe, High confidence	Pass
TC-03	Ambiguous email	Neutral invoice message with words unknown to the model	Uncertain result	Not spam at 50.0% spam probability; Suspicious band, Low confidence	Pass
TC-04	Empty input	Blank text	Error message	HTTP 400	Pass
TC-05	Oversized input	50,001 characters	Error message	HTTP 400	Pass
TC-06	.eml upload	Valid .eml with subject and body	Text and subject extracted	HTTP 200, subject extracted	Pass
TC-07	Wrong file type	.pdf upload	Rejected	HTTP 400, "Invalid file format. Please upload a .txt or .eml file."	Pass
TC-08	Empty file	Empty .txt upload	Rejected	HTTP 400, "The uploaded file is empty."	Pass
TC-09	PDF report	Request report for spam text	PDF returned	HTTP 200, application/pdf	Pass
TC-10	Page availability	Open /, /analyzer, /dashboard, /history, /how-it-works, /about, /health, /api/model-info	All load	All HTTP 200	Pass
TC-11	Unknown route	Open /nope	Not found	HTTP 404	Pass
TC-12	Legacy form endpoint	POST /predict with empty message	Rejected	HTTP 400	Pass

6.3 Evaluation Metrics

With spam as the positive class, the metrics are defined as follows.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1-score = 2 × Precision × Recall / (Precision + Recall)

Table 10: Classifier Performance (computed by train_model.py)

Evaluation	Messages	Accuracy	Precision	Recall	F1-score	TN	FP	FN	TP
20% hold-out	6	83.33%	50.00%	100.00%	66.67%	4	1	0	1
5-fold cross-validation	30	86.67%	90.91%	76.92%	83.33%	16	1	3	10

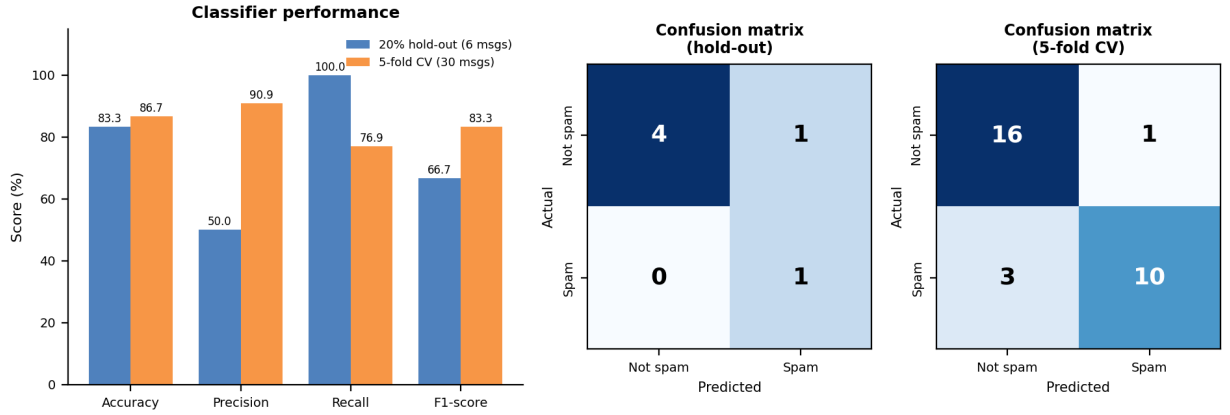


Figure 6.1: Classifier performance and confusion matrices

6.4 Results and Discussion

The application classifies clear spam (prize, urgency, and link wording) as high risk and ordinary conversation as safe, and it explains each result with influential terms, highlighted keywords, and link checks. The rule-based layer adds value beyond the model: for example, a link shortener inside a message is flagged even when the model is unsure.

The cross-validation figures give the better estimate. A precision of 90.91% means most messages flagged as spam were spam, while a recall of 76.92% means about one in four spam messages in the data was missed (3 of 13). The hold-out result rests on only six messages, so a single error moves the accuracy by almost 17 points, and its 50% precision reflects one false positive among two predicted spam messages. These numbers should not be read as production performance.

Test case TC-03 shows the main weakness: words that never appeared in the 30 training messages are ignored, so an unfamiliar email can land near 50%. The interface reports vocabulary coverage and a Low confidence label in this situation instead of presenting false certainty.

Table 11: Result Evaluation Areas

Evaluation area	**Verification approach**	**Outcome**
Spam / safe classification	Sample spam and legitimate emails (TC-01, TC-02)	Classified as expected
Uncertainty handling	Email with unknown vocabulary (TC-03)	Suspicious band, Low confidence
Input validation	Empty, oversized, wrong-type, and empty-file inputs (TC-04 to TC-08)	Rejected with clear errors
Report export	Request PDF report (TC-09)	PDF generated
Availability	All pages and API routes (TC-10, TC-11)	Pages load; unknown route returns 404
Model quality	Hold-out and 5-fold cross-validation	86.67% CV accuracy on a 30-message dataset

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The Email Spam Detector was developed as a web-based prototype that classifies emails with a TF-IDF and Multinomial Naive Bayes pipeline and explains its output through probabilities, a risk score, influential terms, keyword highlighting, link analysis, a dashboard, history, and a PDF report.

The project applies core data-mining ideas: preprocessing, feature extraction, classification, and evaluation with cross-validation. It also shows that a model's prediction can be kept separate from transparent rules so that users can see both. Before any real-world use, the model should be retrained on a much larger dataset and the system strengthened with authentication, header analysis, and stronger link checks.

7.2 Limitations

- The training set has only 30 messages, so metrics and predictions are weak and unfamiliar words are ignored.
- Only message text is analysed; headers, sender authentication, and attachments are not examined.
- Link checks are heuristic and do not use blocklists or visit the link.
- Keyword rules are English-oriented and hand-written.
- History lives in one browser and is lost if browser data is cleared.
- Naive Bayes ignores word order and context.
- The tool is educational and is not a definitive security verdict.

7.3 Future Scope

- Retrain on a large public dataset (for example the SMS Spam Collection or an email corpus) and compare Naive Bayes with logistic regression, SVM, and transformer models.
- Add header analysis (SPF, DKIM, DMARC) and attachment scanning.
- Check links against threat-intelligence or blocklist services.
- Connect directly to a mailbox through IMAP or an email-provider API.

- Add user accounts with a database for history, and a feedback button to retrain from corrections.
- Support more languages and add multilingual keyword rules.
- Deploy the application online with authentication and monitoring.

REFERENCES

[1] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, Morgan Kaufmann.

[2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[5] A. McCallum and K. Nigam, "A comparison of event models for Naive Bayes text classification," *AAAI-98 Workshop on Learning for Text Categorization*, 1998.

[6] T. A. Almeida, J. M. Gómez Hidalgo, and A. Yamakami, "Contributions to the study of SMS spam filtering: new collection and results," *Proceedings of the ACM Symposium on Document Engineering (DocEng)*, 2011.

[7] Flask documentation, https://flask.palletsprojects.com/

[8] ReportLab documentation, https://docs.reportlab.com/

[9] Chart.js documentation, https://www.chartjs.org/docs/

[10] Email Spam Detector (Email Security Analyzer), project source code (Python / Flask), P. Dey, 2026.